UNIVERSIDAD AUTÓNOMA DE BUCARAMANGA

FACULTAD DE INGENIERÍA

PLAN DE DESARROLLO DE SOFTWARE

Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos

Versión 2.0 — Versión definitiva

INGENIERÍA DE SOFTWARE II

PRESENTAN:

Jaime Alejandro Vega Barbosa – U00178766

José Alejandro Téllez Prada – U00060479

David Santiago González Nigrinis – U00179507

DOCENTE:

Daniel Arenas Seeley

PROGRAMA

Ingeniería de Sistemas

Colombia, Bucaramanga, Santander

25 de septiembre de 2026

Declaración de uso de IA generativa: *se empleó Claude (Anthropic) como apoyo para estructurar el documento según la plantilla de Plan de Desarrollo de Software (Rational Unified Process) suministrada por la asignatura, para organizar la estimación por Puntos de Función y COCOMO, para elaborar el cronograma de 8 semanas y los diagramas del documento, y para revisar la redacción técnica. Las recomendaciones de la IA fueron evaluadas críticamente por el equipo; los supuestos de estimación (complejidad de procesos, factores de ajuste, productividad por lenguaje) se declaran explícitamente y quedan sujetos a validación con el docente.*

Historial de revisiones

Versión	Fecha	Descripción del cambio
1.0	16-sep-2026	Primera versión del plan: organización, estimación por Puntos de Función y COCOMO, fases RUP y planes de soporte.
2.0	25-sep-2026	Versión definitiva: metodología de desarrollo (Scrum adaptado dentro de las fases RUP), cronograma de 8 semanas en cuatro sprints de dos semanas, aterrizaje de la estimación al horizonte de 8 semanas y frontera de la aplicación con agentes externos y archivos lógicos (ILF/EIF).

1. Introducción

1.1 Propósito

Este Plan de Desarrollo de Software describe el enfoque de gestión y el proceso técnico que el equipo seguirá para construir la Plataforma Multimodal de Micromovilidad, Logística y Venta de Vehículos Urbanos a partir del primer avance ya entregado. Define fases, iteraciones, estimación de esfuerzo, organización del equipo, mecanismos de control y planes de soporte del proyecto.

1.2 Alcance

Este plan aplica al desarrollo completo del proyecto académico de la asignatura Ingeniería de Software II, tomando como línea base el prototipo funcional urban-mobility-hub (primer avance: landing page, aplicación de mapa y panel de telemetría con datos simulados) y proyectando, en un horizonte de ocho (8) semanas organizado en cuatro sprints de dos semanas, las iteraciones necesarias para incorporar los elementos aún pendientes descritos en el SRS: reconocimiento facial real, integración con hardware Raspberry Pi vía MQTT, pasarela de pagos y persistencia en backend.

1.3 Definiciones, acrónimos y abreviaturas

Se remite al numeral 1.3 del documento SRS_Plataforma_Micromovilidad, que aplica sin modificaciones a este plan.

1.4 Referencias

- SRS_Plataforma_Micromovilidad.docx — Especificación de Requisitos de Software, v2.0.
- Analisis_Riesgos_Plataforma_Micromovilidad.docx — Análisis de Riesgos del Desarrollo, v2.0.
- Repositorio del prototipo: github.com/Joserraldo/urban-mobility-hub.
- Plantilla de Plan de Desarrollo de Software (Rational Unified Process), suministrada por la asignatura.
- Schwaber, K. y Sutherland, J. La Guía de Scrum, 2020.
- Boehm, B. Software Engineering Economics. Prentice Hall, 1981 (modelo COCOMO).

1.5 Resumen del documento

La sección 2 presenta la visión general del proyecto. La sección 3 describe la organización del equipo. La sección 4 desarrolla el proceso de gestión: estimación (incluida la frontera de la aplicación con sus agentes

externos y archivos lógicos, y el aterrizaje al horizonte de 8 semanas), plan de fases y sprints, cronograma de 8 semanas, recursos, presupuesto y mecanismos de monitoreo y control. La sección 5 describe la metodología de desarrollo y los planes técnicos (métodos y herramientas, infraestructura, aceptación). La sección 6 describe los planes de soporte (configuración, evaluación, documentación, calidad y resolución de problemas).

2. Visión general del proyecto

2.1 Propósito, alcance y objetivos del proyecto

El proyecto busca construir una plataforma que integre venta y alquiler de vehículos urbanos (patinetas, bicicletas y drones) con seguimiento en tiempo real y un módulo de seguridad IoT, validando el modelo antes de operar con flota física mediante gemelos digitales. Objetivos del proyecto:

- Entregar el primer avance (portal de negocio, landing page y simulación de flota) — completado.
- Especificar y validar los requisitos funcionales y no funcionales del sistema mediante el SRS.
- Ejecutar la estimación de esfuerzo mediante Puntos de Función y COCOMO, y aterrizarla al equipo real de 3 personas.
- Evolucionar el prototipo hacia la integración de los servicios externos pendientes (reconocimiento facial, pagos, mapas, MQTT/Raspberry Pi).
- Completar el desarrollo en un horizonte de 8 semanas mediante cuatro sprints de dos semanas, con un incremento verificable al cierre de cada sprint.
- Gestionar activamente los riesgos de desarrollo identificados en el Análisis de Riesgos.

2.2 Supuestos y restricciones

- El equipo de desarrollo real está compuesto por exactamente 3 personas durante todo el proyecto.
- No se dispone de vehículos físicos ni de placas Raspberry Pi en operación durante el primer avance; se asume acceso a dicho hardware en fases posteriores.
- El desarrollo restante se ejecuta en un horizonte fijo de 8 semanas (21-sep al 15-nov-2026), con hitos de entrega alineados a los cortes de evaluación definidos por el docente.
- Cada integrante dedica en promedio 15 horas semanales al proyecto, dado que cursa otras asignaturas en paralelo; este valor se usa para calcular la capacidad real del equipo (numeral 4.1.4).
- El stack tecnológico del prototipo (TanStack Start, React 19, Tailwind CSS 4, TypeScript) se mantiene para el resto del proyecto.
- No existe presupuesto monetario real asignado; el presupuesto del proyecto se expresa en esfuerzo (persona-semana) del equipo.

2.3 Entregables del proyecto

Entregable	Descripción	Fecha objetivo
Prototipo — primer avance	Landing, app de mapa y panel de telemetría con flota simulada (ya entregado).	15-sep-2026

Entregable	Descripción	Fecha objetivo
SRS v2.0	Versión definitiva con diagrama de contexto, agentes externos, DFD de nivel 1 y archivos lógicos.	25-sep-2026 (H1)
Análisis de Riesgos v2.0	Riesgos del desarrollo, matriz, plan de SGR y ubicación de los riesgos en el diagrama de contexto.	25-sep-2026 (H1)
Plan de Desarrollo v2.0	Este documento, con metodología y cronograma de 8 semanas.	25-sep-2026 (H1)
Arquitectura y contrato de datos	Backend base, esquema de archivos lógicos, autenticación y desacople de la UI (Sprint 1).	02-oct-2026 (H2)
Prototipo — segundo avance	Compras y reservas persistentes; desbloqueo QR + verificación de casco (Sprint 2).	16-oct-2026 (H3)
Prototipo — tercer avance	Reconocimiento facial, MQTT con simulador de Raspberry Pi y triggers de seguridad (Sprint 3).	30-oct-2026 (H4)
Entrega final	Pasarela de pagos en sandbox, pruebas de aceptación, documentación de cierre y sustentación (Sprint 4).	13-nov-2026 (H5)

2.4 Evolución del Plan de Desarrollo

Este documento se revisará al cierre de cada sprint o ante cambios significativos de alcance, estimación o riesgos. Cada revisión incrementa la versión menor (v2.1, v2.2, …) y se registra en el historial de versiones del documento.

3. Organización del proyecto

3.1 Estructura organizacional

El equipo opera como una célula única de 3 personas sin jerarquía formal; las decisiones técnicas se toman por consenso y el docente actúa como autoridad de revisión y aceptación de los entregables académicos.

Rol	Integrante	Responsabilidad principal
Líder técnico / Arquitectura	David Santiago González Nigrinis	Decisiones de arquitectura, integración (MQTT, APIs), calidad técnica del código.
Líder funcional / Product Owner del grupo	Jaime Alejandro Vega Barbosa	Gestión de requisitos, alcance, relación con el docente, priorización del backlog.
Responsable de planeación y riesgos	José Alejandro Téllez Prada	Estimación, cronograma, seguimiento de riesgos y de la carga de trabajo del equipo.

Los tres integrantes participan en el diseño, la implementación y las pruebas de todos los módulos; los roles anteriores indican responsabilidad de seguimiento, no exclusividad de tareas.

3.2 Interfaces externas

Grupo externo	Contacto interno	Propósito de la interfaz
Docente evaluador (Daniel Arenas Seeley)	Líder funcional	Validación de entregables, retroalimentación sobre alcance y estimación.

Grupo externo	Contacto interno	Propósito de la interfaz
Proveedores de servicios externos (mapas, reconocimiento facial, pagos)	Líder técnico	Integración de APIs de terceros en las fases de construcción.

Las interfaces anteriores son organizacionales. Los agentes externos del sistema —usuario final (AE-01), operador administrativo (AE-02), gemelos digitales (AE-03) y los servicios de reconocimiento facial, pagos y mapas (AE-04 a AE-06)— se especifican en el diagrama de contexto del SRS v2.0 (numeral 2.1) y se retoman en la frontera de la aplicación del numeral 4.1.1.

3.3 Roles y responsabilidades por flujo de trabajo

Flujo de trabajo	Responsable principal	Apoyo
Modelado de requisitos	Jaime Alejandro Vega Barbosa	Equipo completo
Diseño e implementación	David Santiago González Nigrinis	Equipo completo
Pruebas	José Alejandro Téllez Prada	Equipo completo
Gestión de configuración	David Santiago González Nigrinis	—
Gestión de proyecto y riesgos	José Alejandro Téllez Prada	Jaime Alejandro Vega Barbosa

4. Proceso de gestión

4.1 Estimación del proyecto

La estimación se realizó en dos etapas: (1) estimación teórica de tamaño funcional mediante Puntos de Función (IFPUG) y de esfuerzo mediante COCOMO Básico, y (2) aterrizaje de esa estimación a la disponibilidad real del equipo de 3 personas.

4.1.1 Puntos de Función (sin ajustar)

Los procesos funcionales se identificaron a partir de los requisitos funcionales del SRS (RF-01 a RF-19). La Figura 1 delimita la frontera de la aplicación: dentro de ella se ubican los cinco archivos lógicos internos (ILF) que mantiene la plataforma; fuera, los agentes externos que originan las transacciones (EI, EO, EQ) y los tres archivos de interfaz externa (EIF) mantenidos por terceros. El detalle de cada archivo lógico está en el numeral 3.1.11 del SRS.

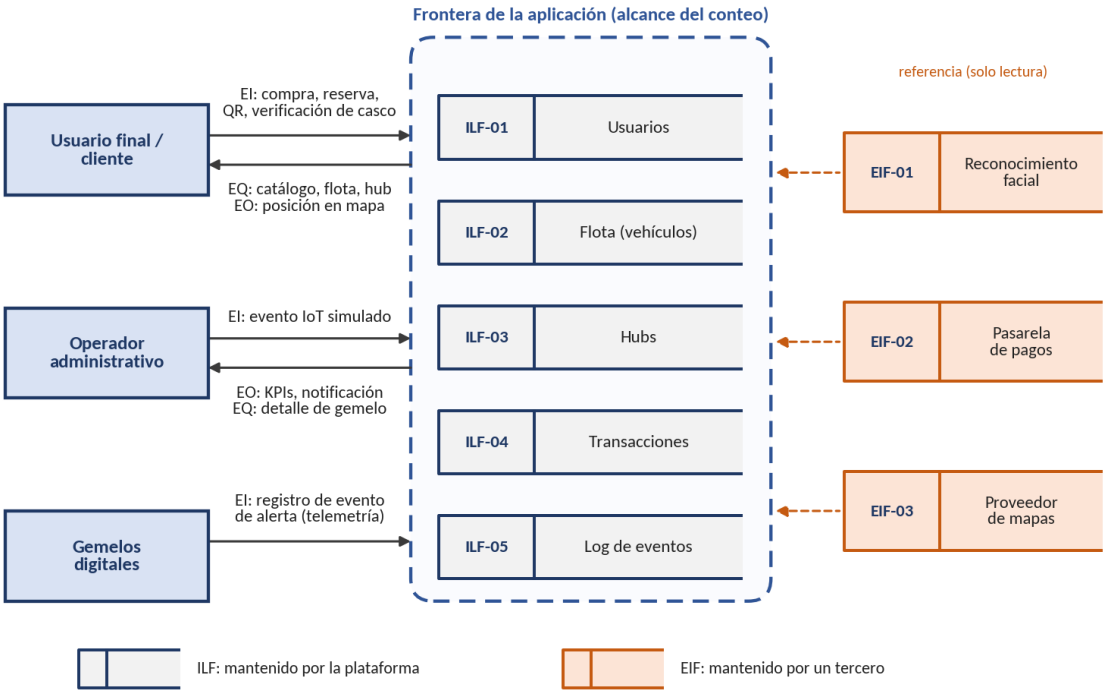


Figura 1. Frontera de la aplicación para el conteo de Puntos de Función: agentes externos, ILF y EIF.

Tipo de componente	Cantidad	Complejidad promedio (PF c/u)	Puntos de función
Archivos lógicos internos (ILF-01 a ILF-05): usuarios, flota, hubs, transacciones (compras y reservas), log de eventos	5	10	50
Archivos de interfaz externa (EIF-01 a EIF-03): verificación biométrica, transacciones de pago, cartografía	3	7	21
Entradas externas (EI): comprar, reservar alquiler, escanear QR, verificar casco, registrar evento de alerta, simular evento IoT	6	4	24
Salidas externas (EO): actualizar posición en mapa, mostrar KPIs de flota, generar notificación de alerta	3	5	15
Consultas externas (EQ): catálogo de venta, flota en alquiler, detalle de hub, detalle de gemelo digital	4	4	16

Puntos de Función sin ajustar (PFSA): 50 + 21 + 24 + 15 + 16 = 126 PF.

4.1.2 Factor de ajuste de valor (VAF)

Se evaluaron las 14 características generales del sistema (GSC) de IFPUG con un grado de influencia promedio de 3 sobre 5 (complejidad media, propia de un sistema con componentes de tiempo real e IoT, pero de alcance académico acotado), para un grado de influencia total (DI) de 42 y un VAF = 0.65 + 0.01 × 42 = 1.07.

Puntos de Función ajustados (PFA): 126 × 1.07 ≈ 135 PF.

4.1.3 Conversión a tamaño de código y esfuerzo (COCOMO Básico)

Con una productividad estimada de 50 líneas de código fuente por punto de función para TypeScript/JavaScript, el tamaño estimado es de 135 × 50 = 6.750 SLOC (6,75 KLOC). Dado que el proyecto combina un dominio conocido por el equipo (aplicación web) con un componente de integración novedoso (IoT/MQTT/reconocimiento facial), se utiliza el modo semi-acoplado (semi-detached) de COCOMO Básico:

Métrica	Fórmula (semi-acoplado)	Resultado
Esfuerzo (persona-mes)	E = 3.0 × (KLOC)^1.12	3.0 × 6.75^1.12 ≈ 24.2 persona-mes
Tiempo de desarrollo (meses)	T = 2.5 × (E)^0.35	2.5 × 24.2^0.35 ≈ 7.4 meses
Personas requeridas	P = E / T	24.2 / 7.4 ≈ 3.3 personas

La estimación teórica converge a un equipo de aproximadamente 3 personas trabajando cerca de 7 a 8 meses para completar el alcance total descrito en el SRS. Este resultado valida, en orden de magnitud, la conformación real del equipo (3 personas), pero evidencia que el horizonte de tiempo académico disponible es considerablemente menor al estimado; en consecuencia, el alcance se libera de forma incremental por avances (ver 4.2), priorizando en cada uno los requisitos de mayor valor y menor riesgo técnico.

Los supuestos de esta estimación (complejidad promedio de los componentes de Puntos de Función, valor del VAF y productividad de 50 SLOC/PF) son ilustrativos y deben validarse y recalibrarse con el docente al cierre de cada sprint, conforme se disponga de datos reales de esfuerzo invertido.

4.1.4 Aterrizaje de la estimación al horizonte de 8 semanas

El proyecto dispone de un horizonte fijo de 8 semanas (≈ 1,85 meses) y de un equipo real de 3 personas con dedicación parcial. La siguiente tabla contrasta el escenario teórico de COCOMO con esa capacidad real:

Concepto	Cálculo	Resultado
Esfuerzo nominal (alcance total del SRS)	COCOMO Básico semi-acoplado (4.1.3)	24,2 persona-mes
Tiempo nominal de desarrollo	T = 2,5 × E^0,35	7,4 meses
Tiempo disponible	8 semanas	≈ 1,85 meses (25% del nominal)
Personas necesarias para el alcance total en 8 semanas	24,2 / 1,85	≈ 13 personas (inviable)
Capacidad real del equipo	3 personas × 8 semanas × 15 h/semana	360 h ≈ 2,4 persona-mes (152 h/pm)
Cobertura del esfuerzo nominal	2,4 / 24,2	≈ 10%

La compresión requerida tampoco es alcanzable agregando personas: Boehm sitúa el límite práctico de compresión del cronograma alrededor del 75% del tiempo nominal (≈ 5,5 meses). En consecuencia, las 8 semanas no buscan completar el alcance total estimado, sino un producto mínimo demostrable que se apoya en tres decisiones:

- **Reutilizar el primer avance:** la interfaz y la simulación de flota ya cubren RF-01, RF-02, RF-04, RF-06 a RF-08 y RF-13 a RF-17 en modo simulado; los sprints se concentran en persistencia, integración y seguridad.

- **Priorizar el alcance (MoSCoW):** se compromete primero lo indispensable para la seguridad del usuario y la demostración del modelo de negocio (tabla siguiente).
- **Recalibrar cada sprint:** el esfuerzo real registrado al cierre de cada sprint ajusta la capacidad del siguiente (riesgos RD3 y RD9).

Prioridad	Alcance comprometido en las 8 semanas	Sprint
Debe (Must)	Contrato de datos y backend base; compras y reservas persistentes (RF-03, RF-05); desbloqueo QR + verificación de casco (RF-09); triggers de seguridad y log auditable (RF-11, RF-12); MQTT con simulador de Raspberry Pi (RF-08, RF-16).	1 a 3
Debería (Should)	Reconocimiento facial con servicio externo y plantillas cifradas (RF-10, RNF-03).	3
Podría (Could)	Pasarela de pagos en modo sandbox para compras y alquileres.	4
No en este periodo (Won't)	Hardware Raspberry Pi físico en operación, pagos reales y operación con flota real.	—

4.2 Plan del proyecto

4.2.1 Plan de fases (WBS)

El proyecto se organiza en cuatro fases RUP, ejecutadas mediante sprints de dos semanas (ver metodología en el numeral 5.1). La Estructura de Descomposición del Trabajo (WBS) de alto nivel es:

Fase	Paquetes de trabajo	Sprint / semanas	Estado
1. Inicio	Modelo de negocio, SRS, análisis de riesgos, estimación FP/COCOMO, plan de desarrollo.	Sprint 0 (08–20 sep)	Completada
2. Elaboración	Arquitectura del prototipo, modelo de datos, landing, app de mapa y panel de telemetría (primer avance); documentación definitiva con diagrama de contexto, DFD y archivos lógicos; contrato de datos y backend base.	Sprint 0 y Sprint 1 (S1–S2)	Primer avance completado; cierre en Sprint 1
3. Construcción	Compras y reservas persistentes, desbloqueo QR + casco, reconocimiento facial, MQTT con simulador de Raspberry Pi, WebSocket, triggers de seguridad y log, pruebas automatizadas de módulos críticos.	Sprints 2 y 3 (S3–S6)	Planificada
4. Transición	Pasarela en sandbox, pruebas de integración y aceptación con el docente, ajustes finales, documentación de cierre y sustentación.	Sprint 4 (S7–S8)	Planificada

4.2.2 Objetivos por sprint

Sprint	Objetivo	Requisitos	Hito
Sprint 0 08–20 sep (previo)	Inicio y primer avance: landing, app de mapa multicapa y panel de telemetría con flota simulada.	RF-01, RF-02, RF-04, RF-06 a RF-08, RF-13 a RF-17 (simulados)	Primer avance (15-sep) — completado

Sprint	Objetivo	Requisitos	Hito
Sprint 1 S1–S2: 21 sep–4 oct	Cerrar la Elaboración: documentación definitiva, contrato de datos entre UI y fuente de datos, backend base con esquema de ILF y autenticación.	RF-18, RF-19, RNF-05, RNF-10	H1 (25-sep), H2 (02-oct)
Sprint 2 S3–S4: 5–18 oct	Compras y reservas persistentes con sus estados; flujo de desbloqueo QR + verificación de casco.	RF-03, RF-05, RF-09, RNF-07	H3 (16-oct)
Sprint 3 S5–S6: 19 oct–1 nov	Reconocimiento facial; broker MQTT con simulador de Raspberry Pi y WebSocket; triggers de seguridad y log auditable.	RF-08, RF-10 a RF-12, RF-16, RNF-01 a RNF-04	H4 (30-oct)
Sprint 4 S7–S8: 2–15 nov	Pasarela de pagos en sandbox; pruebas de integración y aceptación de los cuatro tipos de alerta; cierre y sustentación.	RF-03 y RF-05 (cobro), RNF-06; mitigación de RD7, RD8, RD13	H5 (13-nov)

4.2.3 Entregas (releases)

Release 1 (demo interna, 15-sep, completado): prototipo navegable con datos simulados. Release 2 (beta funcional, 16-oct): compras, reservas y desbloqueo persistentes. Release 3 (30-oct): reconocimiento facial e integración IoT con simulador de Raspberry Pi. Release 4 (entrega final, 13-nov): pasarela de pagos en sandbox y pruebas de aceptación, sujeto a la disponibilidad de los componentes de terceros.

4.2.4 Cronograma del proyecto

El cronograma cubre 8 semanas, del 21 de septiembre al 15 de noviembre de 2026, agrupadas en cuatro sprints de dos semanas. Cada semana tiene un responsable de seguimiento, aunque las tareas se reparten entre los tres integrantes según el Sprint Backlog:

Semana	Fechas	Sprint	Actividades principales	Responsable
S1	21–27 sep	Sprint 1	Planificación del sprint; documentación definitiva (SRS, riesgos, plan); diagrama de contexto, DFD y archivos lógicos; diseño del contrato de datos.	J. Vega
S2	28 sep–4 oct	Sprint 1	Backend base (API REST), esquema de ILF y autenticación; desacople de la UI de los datos simulados; revisión y retrospectiva.	D. González
S3	5–11 oct	Sprint 2	Persistencia de compras y reservas con estados; inicio del flujo de desbloqueo con QR.	D. González
S4	12–18 oct	Sprint 2	Verificación de casco por cámara; pruebas unitarias; demostración del segundo avance (16-oct).	J. Téllez
S5	19–25 oct	Sprint 3	Nivelación en MQTT; broker y simulador de Raspberry Pi; WebSocket; integración del reconocimiento facial.	D. González
S6	26 oct–1 nov	Sprint 3	Triggers de seguridad y log auditable; cifrado de plantillas biométricas; demostración del tercer avance (30-oct).	J. Téllez
S7	2–8 nov	Sprint 4	Pasarela de pagos en sandbox; pruebas de integración de los cuatro tipos de alerta; corrección de defectos.	J. Téllez

Semana	Fechas	Sprint	Actividades principales	Responsable
S8	9–15 nov	Sprint 4	Pruebas de aceptación con el docente; documentación de cierre; sustentación (13-nov); retrospectiva final.	J. Vega

La Figura 2 muestra el mismo cronograma en forma de diagrama de Gantt, con los hitos de control:

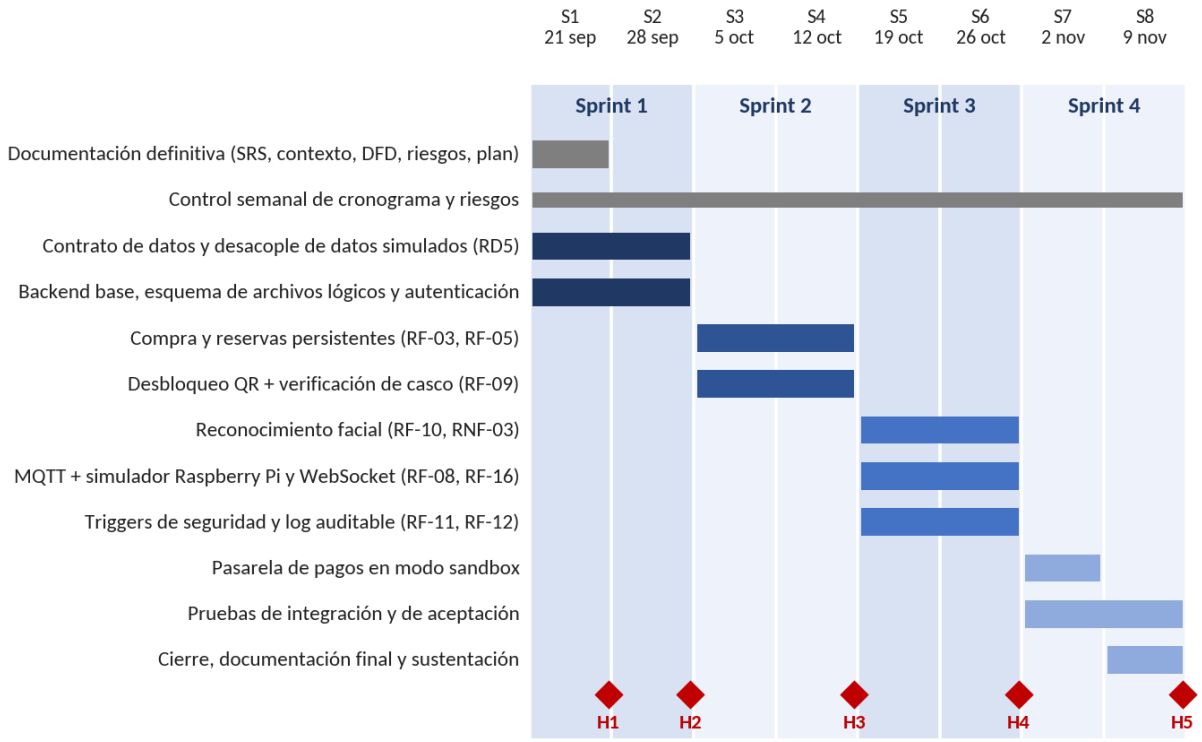


Figura 2. Diagrama de Gantt del cronograma de 8 semanas (sprints, actividades e hitos H1 a H5).

Hito	Descripción	Fecha
H1	Documentación definitiva (SRS, Análisis de Riesgos y Plan de Desarrollo v2.0).	25-sep-2026
H2	Cierre del Sprint 1: arquitectura backend y contrato de datos.	02-oct-2026
H3	Segundo avance: compras, reservas y desbloqueo persistentes.	16-oct-2026
H4	Tercer avance: reconocimiento facial, IoT con simulador y seguridad.	30-oct-2026
H5	Entrega final y sustentación.	13-nov-2026

4.2.5 Recursos del proyecto

Plan de personal: 3 desarrolladores full-stack (los integrantes del equipo), sin roles exclusivos; cada uno participa en requisitos, diseño, implementación y pruebas.

Plan de adquisición de recursos: no se requiere contratación externa; los servicios de terceros (mapas, reconocimiento facial, pagos, broker MQTT) se adoptarán mediante planes gratuitos o de prueba disponibles para proyectos académicos.

Plan de capacitación: sesiones internas de nivelación en protocolo MQTT e integración con Raspberry Pi durante la semana 5 (inicio del Sprint 3), a cargo del líder técnico, dado el riesgo RD4 identificado en el análisis de riesgos.

4.2.6 Presupuesto

Al no existir presupuesto monetario asignado, el presupuesto se expresa en esfuerzo del equipo. Para las 8 semanas la capacidad es de 360 horas-persona (3 integrantes × 8 semanas × 15 h), es decir, 90 horas-persona por sprint, de las cuales se reserva cerca del 15% para los eventos de Scrum y la documentación. Los 24,2 persona-mes estimados por COCOMO se conservan como referencia del alcance total del producto, no como presupuesto de este periodo (ver 4.1.4). Cualquier costo de servicios de terceros que exceda los planes gratuitos disponibles se reportará al docente antes de asumirse.

4.3 Planes de iteración

El detalle de tareas de cada sprint (historias de usuario, tareas técnicas y criterios de aceptación) se gestiona en el Product Backlog y en el Sprint Backlog del equipo, en el tablero del repositorio en GitHub, y se referencia por esta vía; no se duplica en este documento para evitar desactualización.

4.4 Monitoreo y control del proyecto

4.4.1 Plan de gestión de requisitos

Todo cambio sobre los requisitos del SRS debe registrarse, evaluarse en impacto sobre el cronograma y la estimación, y aprobarse por consenso del equipo antes de implementarse, en línea con la mitigación del riesgo RD1.

4.4.2 Plan de control de cronograma

Seguimiento semanal del avance real frente al cronograma de 8 semanas (numeral 4.2.4) y al burndown del sprint; desviaciones mayores a media semana se escalan para replanificación con el docente, en línea con la mitigación del riesgo RD3.

4.4.3 Plan de control de presupuesto

Seguimiento del esfuerzo real (horas/persona-semana) invertido por sprint frente a la capacidad planificada de 90 horas-persona y a lo estimado en COCOMO, para recalibrar la estimación de las iteraciones siguientes (riesgo RD9).

4.4.4 Plan de control de calidad

Cada módulo crítico (desbloqueo, seguridad, reconocimiento facial) debe contar con casos de prueba antes de considerarse cerrado; no se libera un sprint con pruebas pendientes en estos módulos (riesgos RD7, RD8, RD13).

4.4.5 Plan de reportes

Reporte interno semanal de avance entre los integrantes del equipo, y reporte formal al docente en la revisión de cada sprint, incluyendo estado de riesgos.

4.4.6 Plan de medición

Métricas de seguimiento: porcentaje de requisitos funcionales implementados sobre el total del SRS, esfuerzo real vs. planificado por sprint, velocidad del equipo (puntos de historia por sprint), número de defectos abiertos en módulos críticos, y estado de los riesgos priorizados (probabilidad/impacto residual).

4.5 Plan de gestión de riesgos

Se incorpora por referencia el documento Analisis_Riesgos_Plataforma_Micromovilidad.docx, que contiene las categorías de riesgo, la lista completa de riesgos del desarrollo (RD1–RD13), la matriz de priorización y el plan de Supervisión, Reducción y Gestión (SGR) para los riesgos críticos RD3, RD5, RD7, RD8 y RD11.

4.6 Plan de cierre

Al finalizar la fase de Transición se realizará: sustentación del proyecto ante el docente, entrega de la documentación final (SRS, análisis de riesgos, plan de desarrollo y DFD actualizados a su última versión), archivo del código fuente en el repositorio y una retrospectiva interna del equipo sobre la precisión de la estimación inicial frente al esfuerzo real invertido.

5. Planes del proceso técnico

5.1 Metodología de desarrollo (caso de desarrollo)

5.1.1 Enfoque adoptado

El equipo adopta Scrum adaptado a un equipo académico de tres personas, ejecutado dentro de las cuatro fases de RUP que exige la plantilla de la asignatura. RUP aporta la estructura de fases y los artefactos de documentación (SRS, plan, riesgos, DFD); Scrum aporta el ritmo de trabajo: sprints de duración fija, un backlog priorizado y un incremento verificable al final de cada sprint. Se eligió esta combinación por tres razones:

- **Horizonte fijo:** con 8 semanas disponibles, los sprints de dos semanas (timeboxing) obligan a priorizar y dan cuatro puntos de control con el docente.
- **Incertidumbre técnica:** la integración con MQTT, reconocimiento facial y pagos tiene riesgos altos (RD4, RD7, RD12); las iteraciones cortas permiten detectar problemas temprano y replanificar.
- **Equipo pequeño:** tres personas no justifican una ceremonia pesada; los eventos de Scrum se reducen en frecuencia y duración (5.1.3).

5.1.2 Roles

Rol	Integrante	Responsabilidades en el proceso
Product Owner	Jaime Alejandro Vega Barbosa	Mantiene y prioriza el Product Backlog a partir del SRS; acepta o rechaza los incrementos; es el enlace con el docente.
Scrum Master	José Alejandro Téllez Prada	Facilita los eventos, vigila el cronograma y los riesgos, elimina impedimentos y mide la velocidad del equipo.
Equipo de desarrollo	Los tres integrantes (liderazgo técnico: David Santiago González Nigrinis)	Diseñan, implementan y prueban los incrementos; estiman las historias de usuario; el líder técnico decide la arquitectura y la integración.
Interesado principal	Docente Daniel Arenas Seeley	Valida los incrementos en las revisiones de sprint que coinciden con los hitos H3 a H5.

5.1.3 Eventos

Evento	Frecuencia y duración	Propósito
Sprint	2 semanas (4 en total)	Contenedor de trabajo con un objetivo y un incremento verificable.
Planificación del sprint	Lunes de la primera semana del sprint, 1 h	Definir el objetivo del sprint y seleccionar las historias del backlog según la capacidad (90 horas-persona).
Sincronización del equipo	Lunes, miércoles y viernes, 15 min (virtual)	Reemplaza la reunión diaria: avance, próximos pasos e impedimentos.
Refinamiento del backlog	Semanal, 30 min	Detallar y estimar las historias del siguiente sprint.
Revisión del sprint	Viernes de la segunda semana, 45 min	Demostrar el incremento; con el docente en los hitos H3, H4 y H5.
Retrospectiva	Después de la revisión, 30 min	Acordar mejoras concretas de la forma de trabajo para el siguiente sprint.

5.1.4 Artefactos

- **Product Backlog:** requisitos RF y RNF del SRS expresados como historias de usuario y priorizados con MoSCoW (4.1.4).
- **Sprint Backlog:** historias y tareas comprometidas en el sprint, en un tablero con columnas Por hacer, En progreso, En revisión y Hecho.
- **Incremento:** versión del prototipo desplegada en el entorno de demostración al cierre de cada sprint.
- **Burndown del sprint:** trabajo restante por día, usado en el control de cronograma (4.4.2).

5.1.5 Definición de terminado

Una historia de usuario se considera terminada cuando cumple todas las condiciones siguientes:

- El código está integrado en la rama principal mediante un pull request revisado por otro integrante.
- ESLint no reporta errores y los casos de prueba asociados al requisito pasan; en los módulos críticos (desbloqueo, seguridad, reconocimiento facial) las pruebas son automatizadas.
- El comportamiento se verificó contra el requisito correspondiente del SRS.
- El incremento está desplegado en el entorno de demostración y la documentación afectada (README, SRS) está actualizada.

5.2 Métodos, herramientas y técnicas

Ámbito	Herramienta / técnica
Control de versiones	Git / GitHub (repositorio urban-mobility-hub).
Framework y lenguaje	TanStack Start, React 19, TypeScript.
Estilos e interfaz	Tailwind CSS 4, shadcn/ui.
Calidad de código	ESLint, Prettier.
Estimación	Puntos de Función (IFPUG) y COCOMO Básico.

Ámbito	Herramienta / técnica
Gestión de riesgos	Matriz probabilidad/impacto y SGR (ver Análisis de Riesgos).

5.3 Plan de infraestructura

Desarrollo local con Bun/Node.js; despliegue de build de producción orientado a Cloudflare (Nitro), según configuración del repositorio. La integración con hardware Raspberry Pi se realizará inicialmente sobre un simulador de broker MQTT antes de conectar hardware físico.

5.4 Plan de aceptación del producto

Cada sprint se considera aceptado cuando: (a) los requisitos funcionales comprometidos para ese sprint están implementados, verificados contra el SRS y cumplen la definición de terminado (5.1.5), (b) no existen defectos abiertos de severidad alta en los módulos críticos de seguridad, y (c) el docente valida la entrega en el corte correspondiente.

6. Planes del proceso de soporte

6.1 Plan de gestión de configuración

Todo el código fuente se versiona en el repositorio Git urban-mobility-hub sobre la rama principal, con commits descriptivos por funcionalidad. Los documentos de gestión (SRS, riesgos, plan de desarrollo) se versionan de forma independiente con numeración v1.x.

6.2 Plan de evaluación

Se realizarán revisiones internas de código y de diseño al cierre de cada sprint, enfocadas prioritariamente en los módulos de seguridad (desbloqueo, trigger de alertas) por su criticidad identificada en el análisis de riesgos.

6.3 Plan de documentación

La documentación del proyecto (SRS, análisis de riesgos, plan de desarrollo, DFD) se mantiene en la carpeta de curso del equipo y se referencia desde el README del repositorio, sin versionarse dentro del código fuente por encontrarse en revisión activa con el docente.

6.4 Plan de aseguramiento de calidad

Uso de linting automático (ESLint) en cada cambio, pruebas manuales guiadas por los requisitos del SRS antes de cada entrega, y pruebas dirigidas de los cuatro tipos de alerta IoT (sin casco, caída, GPS, batería) antes de dar por cerrado el módulo de seguridad.

6.5 Plan de resolución de problemas

Los defectos se registran como incidencias en el repositorio (GitHub Issues), se priorizan según si afectan módulos críticos de seguridad y se resuelven antes de iniciar el siguiente sprint cuando su severidad es alta.

6.6 Plan de gestión de subcontratistas

No aplica: el proyecto no involucra subcontratistas; todo el desarrollo lo realiza el equipo de 3 integrantes.

6.7 Plan de mejora de procesos

Al cierre de cada sprint el equipo realiza la retrospectiva (5.1.3) para ajustar la forma de trabajo (comunicación entre subequipos, división de tareas), en respuesta directa al riesgo RD10 identificado en el análisis de riesgos.

7. Planes adicionales

No se identifican planes adicionales exigidos por contrato o normativa externa para este proyecto académico.

8. Anexos

- Anexo A: SRS_Plataforma_Micromovilidad.docx (v2.0).
- Anexo B: Analisis_Riesgos_Plataforma_Micromovilidad.docx (v2.0).
- Anexo C: Repositorio del prototipo funcional (github.com/Joserraldo/urban-mobility-hub).